

```
# -*- coding: utf-8 -*-  
"""
```

```
Created on Mon Apr 13 23:10:45 2026
```

```
@author: Manish  
"""
```

```
#
```

```
=====
```

```
=====
```

```
# CE-608 : Ecohydrology - CMIP6 Climate Analysis  
# Student : Manish Bhagat | Roll No : 25D0294  
# Country : Canada  
# Model : IPSL-CM6A-LR  
# Scenario: SSP245  
# Script : Spyder (.py)  
#
```

```
=====
```

```
=====
```

```
# %% SECTION 1 - IMPORT LIBRARIES
```

```
#
```

```
=====
```

```
=====
```

```
import xarray      as xr      # reads and handles NetCDF (.nc) climate  
data files  
import numpy       as np      # fast numerical operations (arrays, maths)  
import matplotlib.pyplot as plt # draws all plots and figures  
import matplotlib.ticker as mticker # controls tick marks on map axes  
import cartopy.crs as ccrs     # map projections (how Earth is  
flattened)  
import cartopy.feature as cfeature # adds coastlines, borders, lakes  
to maps  
from scipy import stats          # linear regression for trend  
calculation  
import warnings                  # library to manage warning  
messages  
warnings.filterwarnings('ignore') # suppress harmless warnings  
(keep console clean)
```

```
print("✓ All libraries imported successfully.")
```

```
#
```

```
=====
```

```
=====
```

```
# %% SECTION 2 - FILE PATHS
```

```
#
```

```
=====
```

```
=====
```

```
# These four variables tell Python where your downloaded NetCDF files  
are.  
# r"..." → raw string: backslashes in Windows paths work correctly.  
# Your files are in E:\eco_data\ - only change this if you move them.
```

```

tas_hist_path = r"E:\eco_data\tas_day_IPSL-CM6A-
LR_historical_r1i1p1f1_gr_18500101-20141231.nc"
# ↑ Temperature | HISTORICAL experiment | 1850-01-01 to 2014-12-31

tas_ssp_path = r"E:\eco_data\tas_day_IPSL-CM6A-
LR_ssp245_r1i1p1f1_gr_20150101-21001231.nc"
# ↑ Temperature | SSP245 future scenario | 2015-01-01 to 2100-12-31

pr_hist_path = r"E:\eco_data\pr_day_IPSL-CM6A-
LR_historical_r1i1p1f1_gr_18500101-20141231.nc"
# ↑ Precipitation | HISTORICAL experiment | 1850-01-01 to 2014-12-31

pr_ssp_path = r"E:\eco_data\pr_day_IPSL-CM6A-
LR_ssp245_r1i1p1f1_gr_20150101-21001231.nc"
# ↑ Precipitation | SSP245 future scenario | 2015-01-01 to 2100-12-31

print("✓ File paths set.")

#
=====
====
# %% SECTION 3 – CANADA BOUNDING BOX AND COORDINATE SETUP
#
=====
====
#
# Canada's geographic extent:
#   South = 42°N (Lake Erie shoreline, Ontario)
#   North = 84°N (Ellesmere Island, Nunavut)
#   West  = 141°W (Yukon/Alaska border)
#   East  = 52°W (Newfoundland coast)
#
# IMPORTANT – LONGITUDE CONVENTION IN IPSL-CM6A-LR:
# The IPSL model stores longitudes from 0° to 360° (NOT -180° to +180°).
# In the 0-360 system, Western-Hemisphere longitudes are converted as:
#   standard = 360 + negative_value
# So: -141° → 360 - 141 = 219°
#   -52° → 360 - 52 = 308°
#
# Cartopy's set_extent() always uses the -180/+180 standard, so we keep
# a separate extent_cartopy variable for the map view.

lon_min = 219.0    # Canada west boundary in 0-360 convention (= 141°W)
lon_max = 308.0    # Canada east boundary in 0-360 convention (= 52°W)
lat_min = 42.0     # Canada south boundary (°N)
lat_max = 84.0     # Canada north boundary (°N)

# extent used inside the data subsetting (.sel)
extent_data = [lon_min, lon_max, lat_min, lat_max]

# extent used by Cartopy to set the map view (always -180 to +180 for
Cartopy)

```

```

extent_cartopy = [-141, -52, 42, 84]    # [west°, east°, south°N, north°N]

print(f"✓ Canada bounding box set: lon {lon_min}-{lon_max}° (0-360),
lat {lat_min}-{lat_max}°N")

#
=====
====
# %% SECTION 4 - LOAD DATA, SUBSET TO CANADA, CONVERT UNITS
#
=====
====
#
# STEP A - LOAD
#   xr.open_dataset(path) opens the NetCDF file and reads its metadata.
#   ['tas'] / ['pr']      extracts just the variable we need (not the
whole file).
#
# STEP B - SUBSET (.sel)
#   .sel(lon=slice(min, max), lat=slice(min, max))
#   selects only the grid points that fall inside Canada's bounding box.
#   Without this, we'd be processing the entire global grid - very slow.
#   slice(a, b) means "select everything between a and b".
#   The IPSL 'gr' grid has lat in ASCENDING order (-90 → +90), so
#   slice(lat_min, lat_max) = slice(42, 84) works directly.
#
# STEP C - UNIT CONVERSION
#   Temperature (tas):
#   Model stores in Kelvin (K). We subtract 273.15 to get Celsius
(°C).
#   Reason: 0 K = absolute zero (-273.15 °C). 0 °C = freezing point of
water.
#
#   Precipitation (pr):
#   Model stores in kg m-2 s-1 = mm s-1 (water density = 1000 kg/m3)
#   We multiply by 86400 (seconds per day) to get mm per day.
#   86400 = 24 hours × 60 minutes × 60 seconds

print("\nLoading and subsetting data for Canada... (may take 1-3
minutes)")

# --- TEMPERATURE - HISTORICAL ---
tas_hist = (xr.open_dataset(tas_hist_path)    # open the NetCDF file
            ['tas']                          # extract the temperature
variable
            .sel(lon=slice(lon_min, lon_max), # keep only Canada
longitudes
            lat=slice(lat_min, lat_max)) # keep only Canada
latitudes
            - 273.15)                        # convert Kelvin →
Celsius
print(" ✓ tas historical loaded and converted to °C")

```

```

# --- TEMPERATURE - SSP245 ---
tas_ssp = (xr.open_dataset(tas_ssp_path)
            ['tas']
            .sel(lon=slice(lon_min, lon_max),
                  lat=slice(lat_min, lat_max))
            - 273.15) # convert Kelvin →
Celsius
print(" ✓ tas SSP245 loaded and converted to °C")

# --- PRECIPITATION - HISTORICAL ---
pr_hist = (xr.open_dataset(pr_hist_path)
            ['pr']
            .sel(lon=slice(lon_min, lon_max),
                  lat=slice(lat_min, lat_max))
            * 86400) # convert kg/m²/s →
mm/day
print(" ✓ pr historical loaded and converted to mm/day")

# --- PRECIPITATION - SSP245 ---
pr_ssp = (xr.open_dataset(pr_ssp_path)
            ['pr']
            .sel(lon=slice(lon_min, lon_max),
                  lat=slice(lat_min, lat_max))
            * 86400) # convert kg/m²/s →
mm/day
print(" ✓ pr SSP245 loaded and converted to mm/day")

# --- CONFIRMATION PRINTOUT ---
print(f"\n Historical period : {str(tas_hist.time.values[0])[:10]} → "
      f"{str(tas_hist.time.values[-1])[:10]}")
print(f" SSP245 period : {str(tas_ssp.time.values[0])[:10]} → "
      f"{str(tas_ssp.time.values[-1])[:10]}")
print(f" Grid over Canada : {tas_hist.shape[1]} lat pts × "
      f"{tas_hist.shape[2]} lon pts")
print(" Units: tas → °C | pr → mm/day")

#
=====
====
# %% SECTION 5 - REUSABLE HELPER: SPATIAL MAP PLOT FUNCTION
#
=====
====
# We define this function once so all spatial map plots use the same
# formatting without repeating the same 20 lines of code each time.
#
# A function works like a recipe:
# def function_name(ingredient1, ingredient2, ...):
#     instructions
# We "call" it later by:
# function_name(actual_data, "Title", ...)

```

```

#
# PlateCarree projection = simplest lat/lon grid, lines are straight.
# For Canada's large area, this shows a slight distortion at high
latitudes,
# but it matches the coordinate system of the data exactly.

def spatial_plot(data, title, cmap, cbar_label, vmin=None, vmax=None,
extend='both'):
    """
    Draw a coloured 2-D map of a climate variable over Canada.

    Parameters
    -----
    data      : 2-D xarray DataArray (lat × lon), already time-
averaged.
    title     : String – figure title.
    cmap      : Matplotlib colormap name (e.g. 'RdYlBu_r', 'YlGnBu',
'BrBG').
    cbar_label : String – label for the colour bar.
    vmin, vmax : Optional floats – fix the colour scale. None = auto.
    extend    : Colour bar arrow style – 'both', 'min', 'max', or
'neither'.
    """
    fig, ax = plt.subplots(
        figsize=(11, 7),                                # figure size in inches
        subplot_kw={'projection': ccrs.PlateCarree()})    # map projection
    for the axes
    )

    data.plot(                                           # xarray's built-in plot
method
        ax=ax,                                           # draw on the map axes we
created above
        transform=ccrs.PlateCarree(),                   # our data coordinates
are lat/lon degrees
        cmap=cmap,                                       # which colour palette to
use
        vmin=vmin, vmax=vmax,                           # colour scale limits
        add_colorbar=True,                             # show a colour bar next
to the map
        cbar_kwargs={
            'label' : cbar_label,                       # colour bar text label
            'shrink': 0.65,                             # makes colour bar
shorter (cleaner look)
            'extend': extend,                           # triangle arrows if data
exceeds limits
            'pad'   : 0.02                              # gap between map and
colour bar
        }
    )

    # Add geographic reference features on top of the colour field

```

```

        ax.add_feature(cfeature.COASTLINE, linewidth=0.8) #
ocean/land edge
        ax.add_feature(cfeature.BORDERS, linewidth=0.7, linestyle='--') #
country borders
        ax.add_feature(cfeature.STATES, linewidth=0.3, linestyle=':') #
province boundaries
        ax.add_feature(cfeature.LAKES, linewidth=0.4, alpha=0.5) #
Great Lakes etc.
        ax.add_feature(cfeature.OCEAN, color='lightcyan', alpha=0.25) #
light ocean tint

# Crop the map view to Canada only (hides the rest of the globe)
ax.set_extent(extent_cartopy, crs=ccrs.PlateCarree())

# Draw latitude/longitude reference grid lines
gl = ax.gridlines(draw_labels=True, linewidth=0.3,
                  alpha=0.5, linestyle='--', color='gray')
gl.top_labels = False # no labels along the top edge
gl.right_labels = False # no labels along the right edge
gl.xlocator = mticker.FixedLocator([-140, -120, -100, -80, -60]) #
lon tick positions
gl.ylocator = mticker.FixedLocator([45, 55, 65, 75]) #
lat tick positions

ax.set_title(title, fontsize=13, fontweight='bold', pad=12)
plt.tight_layout() # automatically adjust spacing so labels don't
overlap
plt.show() # display the plot in Spyder's Plots pane

# Shared month names list – used by multiple plots below
months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
          'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']

print("✓ Helper function defined. Ready to generate plots.\n")

#
=====
====
# %% SECTION 6 – PLOT 1: DAILY CLIMATOLOGY – TEMPERATURE
#
=====
====
#
# WHAT IS A DAILY CLIMATOLOGY?
# For 165 years of daily data (1850–2014), we have ~60,225 daily values.
# groupby('time.dayofyear') groups ALL values that share the same day-of-
year:
# → Group 1 = all January 1st values across every year (165 values)
# → Group 2 = all January 2nd values across every year (165 values)
# → ... up to Group 365 = all December 31st values
# .mean(dim=['time','lat','lon'])
# → For each group, average over ALL years AND all Canada grid points

```

```

#   → Result = 365 numbers = the "typical" seasonal cycle of Canada
#
# OUTPUT: A line plot with Day-of-Year on X-axis, Temperature on Y-axis.
#         Historical line (blue) vs SSP245 line (red).
#         You will see Canada's cold winters and warm summers as a wave
#         shape.

print("Generating Plot 1: Daily Climatology – Temperature")

# Group by calendar day (1-365), average over all years AND all grid
# cells
tas_hist_daily_clim =
tas_hist.groupby('time.dayofyear').mean(dim=['time', 'lat', 'lon'])
tas_ssp_daily_clim = tas_ssp.groupby('time.dayofyear').mean(dim=['time',
'lat', 'lon'])
# Each result is now a 365-element array (one average per day of year)

fig, ax = plt.subplots(figsize=(12, 5)) # create figure (12 wide, 5
tall inches)
ax.plot(tas_hist_daily_clim.dayofyear, tas_hist_daily_clim,
        label='Historical (1850-2014)', linewidth=0.9, color='steelblue')
# ↑ X = day number 1-365, Y = area-averaged temperature for that day
ax.plot(tas_ssp_daily_clim.dayofyear, tas_ssp_daily_clim,
        label='SSP245 (2015-2100)', linewidth=0.9, color='tomato')

ax.axhline(0, color='black', linewidth=0.5, alpha=0.4, linestyle='--') #
0°C reference line
ax.set_xlabel('Day of Year (1 = Jan 1, 365 = Dec 31)', fontsize=11)
ax.set_ylabel('Temperature (°C)', fontsize=11)
ax.set_title('Daily Climatology – Temperature (Area-Averaged over
Canada)', fontsize=13, fontweight='bold')
ax.legend(fontsize=10)
ax.grid(True, alpha=0.25) # light background grid for readability
plt.tight_layout()
# plt.savefig(r"E:\eco_data\plots\01_daily_clim_tas.png", dpi=150,
bbox_inches='tight')
plt.show()

#
=====
====
# %% SECTION 7 – PLOT 2: DAILY CLIMATOLOGY – PRECIPITATION
#
=====
====
#
# Same logic as Plot 1 but for precipitation.
# OUTPUT: Shows Canada's wet/dry seasonal cycle.
#         You should see higher precipitation in summer (June–August).

print("Generating Plot 2: Daily Climatology – Precipitation")

```

```

pr_hist_daily_clim = pr_hist.groupby('time.dayofyear').mean(dim=['time',
'lat', 'lon'])
pr_ssp_daily_clim = pr_ssp.groupby('time.dayofyear').mean(dim=['time',
'lat', 'lon'])
# Each result = 365 values, one per day of year, averaged over all years
and Canada

fig, ax = plt.subplots(figsize=(12, 5))
ax.plot(pr_hist_daily_clim.dayofyear, pr_hist_daily_clim,
        label='Historical (1850-2014)', linewidth=0.9, color='steelblue')
ax.plot(pr_ssp_daily_clim.dayofyear, pr_ssp_daily_clim,
        label='SSP245 (2015-2100)', linewidth=0.9, color='tomato')
ax.set_xlabel('Day of Year (1 = Jan 1, 365 = Dec 31)', fontsize=11)
ax.set_ylabel('Precipitation (mm/day)', fontsize=11)
ax.set_title('Daily Climatology - Precipitation (Area-Averaged over
Canada)', fontsize=13, fontweight='bold')
ax.legend(fontsize=10)
ax.grid(True, alpha=0.25)
plt.tight_layout()
# plt.savefig(r"E:\eco_data\plots\02_daily_clim_pr.png", dpi=150,
bbox_inches='tight')
plt.show()

```

```

#
=====
====
# %% SECTION 8 - PLOT 3: MONTHLY CLIMATOLOGY - TEMPERATURE
#
=====
====
#
# WHAT IS A MONTHLY CLIMATOLOGY?
# Same idea as daily but grouped by calendar MONTH (1-12).
# groupby('time.month') → 12 groups (Jan, Feb, ..., Dec)
# .mean(dim=['time','lat','lon']) → 12 numbers = mean annual cycle
#
# OUTPUT: A line plot with 12 months on X-axis.
#         Shows the mean temperature for each month.
#         Canada's January will be very cold (below -10°C).

print("Generating Plot 3: Monthly Climatology - Temperature")

tas_hist_monthly_clim = tas_hist.groupby('time.month').mean(dim=['time',
'lat', 'lon'])
tas_ssp_monthly_clim = tas_ssp.groupby('time.month').mean(dim=['time',
'lat', 'lon'])
# Each result = 12 values (one average per calendar month)

fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(range(1, 13), tas_hist_monthly_clim,
        'o-', label='Historical (1850-2014)', linewidth=2, markersize=7,
color='steelblue')
# 'o-' = circles at each month connected by a line

```



```

ax.plot(range(1, 13), tas_ssp_monthly_clim,
        's-', label='SSP245 (2015-2100)', linewidth=2, markersize=7,
        color='tomato')
# 's-' = squares at each month connected by a line
ax.axhline(0, color='black', linewidth=0.5, alpha=0.4, linestyle='--') #
0°C reference
ax.set_xticks(range(1, 13)) # place tick at every month number
ax.set_xticklabels(months) # replace numbers with month names
ax.set_ylabel('Temperature (°C)', fontsize=11)
ax.set_title('Monthly Climatology – Temperature (Area-Averaged over
Canada)', fontsize=13, fontweight='bold')
ax.legend(fontsize=10)
ax.grid(True, alpha=0.25)
plt.tight_layout()
# plt.savefig(r"E:\eco_data\plots\03_monthly_clim_tas.png", dpi=150,
bbox_inches='tight')
plt.show()

```

```

#
=====
====
# %% SECTION 9 – PLOT 4: MONTHLY CLIMATOLOGY – PRECIPITATION
#
=====
====
#
# Bar chart is better for monthly precipitation – each month is a
distinct
# category, not a continuous wave like temperature.
# Two bars side-by-side per month: blue = historical, red = SSP245.
# Offset by ±0.15 so the bars don't overlap each other.
#
# OUTPUT: Which months are wetter/drier in the future vs the past.

print("Generating Plot 4: Monthly Climatology – Precipitation")

pr_hist_monthly_clim = pr_hist.groupby('time.month').mean(dim=['time',
'lat', 'lon'])
pr_ssp_monthly_clim = pr_ssp.groupby('time.month').mean(dim=['time',
'lat', 'lon'])

fig, ax = plt.subplots(figsize=(10, 5))
ax.bar(np.arange(1, 13) - 0.15, pr_hist_monthly_clim, width=0.3,
        label='Historical (1850-2014)', color='steelblue', alpha=0.85,
        edgecolor='white')
# np.arange(1,13)-0.15 → shift bars 0.15 units LEFT so two bars fit per
month
ax.bar(np.arange(1, 13) + 0.15, pr_ssp_monthly_clim, width=0.3,
        label='SSP245 (2015-2100)', color='tomato', alpha=0.85,
        edgecolor='white')
# +0.15 → shift bars 0.15 units RIGHT
ax.set_xticks(range(1, 13))
ax.set_xticklabels(months)

```

```

ax.set_ylabel('Precipitation (mm/day)', fontsize=11)
ax.set_title('Monthly Climatology – Precipitation (Area-Averaged over
Canada)', fontsize=13, fontweight='bold')
ax.legend(fontsize=10)
ax.grid(True, alpha=0.25, axis='y')    # horizontal grid lines only (y-
axis)
plt.tight_layout()
# plt.savefig(r"E:\eco_data\plots\04_monthly_clim_pr.png", dpi=150,
bbox_inches='tight')
plt.show()

#
=====
====
# %% SECTION 10 – PLOT 5: YEARLY CLIMATOLOGY – TEMPERATURE
#
=====
====
#
# WHAT IS A YEARLY CLIMATOLOGY (TIME SERIES)?
# Instead of the seasonal cycle, we look at LONG-TERM CHANGE year by
year.
# groupby('time.year').mean('time') → collapses all 365 daily values in
each
#   year into a single annual mean → one number per year (165 for
historical)
# .mean(dim=['lat','lon']) → average over all Canada grid points
# → Result: one temperature value per calendar year (e.g. -2.3°C for
1850)
#
# OUTPUT: A time series from 1850 to 2100, showing year-by-year
variability
#           and the long-term warming trend from historical through SSP245.

print("Generating Plot 5: Yearly Climatology – Temperature")

tas_hist_yearly = tas_hist.groupby('time.year').mean(dim=['time', 'lat',
'lon'])
tas_ssp_yearly = tas_ssp.groupby('time.year').mean(dim=['time', 'lat',
'lon'])
# Result: 165 values for historical (1850-2014), 86 values for SSP245
(2015-2100)

fig, ax = plt.subplots(figsize=(14, 5))
ax.plot(tas_hist_yearly.year, tas_hist_yearly,
        label='Historical (1850-2014)', linewidth=0.9, color='steelblue')
ax.plot(tas_ssp_yearly.year, tas_ssp_yearly,
        label='SSP245 (2015-2100)', linewidth=0.9, color='tomato')
ax.set_xlabel('Year', fontsize=11)
ax.set_ylabel('Temperature (°C)', fontsize=11)
ax.set_title('Yearly Mean Temperature – Canada (1850-2100)', fontsize=13,
fontweight='bold')
ax.legend(fontsize=10)

```

```

ax.grid(True, alpha=0.25)
plt.tight_layout()
# plt.savefig(r"E:\eco_data\plots\05_yearly_clim_tas.png", dpi=150,
bbox_inches='tight')
plt.show()

#
=====
====
# %% SECTION 11 – PLOT 6: YEARLY CLIMATOLOGY – PRECIPITATION
#
=====
====
#
# OUTPUT: Year-by-year precipitation time series from 1850 to 2100.
#         Canada's precipitation varies substantially year to year.

print("Generating Plot 6: Yearly Climatology – Precipitation")

pr_hist_yearly = pr_hist.groupby('time.year').mean(dim=['time', 'lat',
'lon'])
pr_ssp_yearly  = pr_ssp.groupby('time.year').mean(dim=['time', 'lat',
'lon'])

fig, ax = plt.subplots(figsize=(14, 5))
ax.plot(pr_hist_yearly.year, pr_hist_yearly,
        label='Historical (1850-2014)', linewidth=0.9, color='steelblue')
ax.plot(pr_ssp_yearly.year, pr_ssp_yearly,
        label='SSP245 (2015-2100)', linewidth=0.9, color='tomato')
ax.set_xlabel('Year', fontsize=11)
ax.set_ylabel('Precipitation (mm/day)', fontsize=11)
ax.set_title('Yearly Mean Precipitation – Canada (1850-2100)',
fontsize=13, fontweight='bold')
ax.legend(fontsize=10)
ax.grid(True, alpha=0.25)
plt.tight_layout()
# plt.savefig(r"E:\eco_data\plots\06_yearly_clim_pr.png", dpi=150,
bbox_inches='tight')
plt.show()

#
=====
====
# %% SECTION 12 – PLOTS 7-10: SPATIAL MEAN MAPS
#
=====
====
#
# WHAT IS A SPATIAL MEAN MAP?
# Instead of averaging over SPACE (like in Sections 6-11), here we
average
# over TIME – collapsing all days into one number per grid cell.

```

```

# .mean(dim='time') → produces a 2D field (lat × lon) showing WHERE in
# Canada it is typically warm/cold or wet/dry.
#
# OUTPUT:
#   Plot 7  - Historical mean temperature map (1850-2014)
#             You see Canada's north is colder than the south.
#   Plot 8  - SSP245 mean temperature map (2015-2100)
#             Future map; should be overall warmer than Plot 7.
#   Plot 9  - Historical mean precipitation map (1850-2014)
#             Pacific coast and southeast are wetter.
#   Plot 10 - SSP245 mean precipitation map (2015-2100)

print("Generating Plots 7-10: Spatial mean maps")

# Time-average: collapse all time steps → 2D (lat × lon) fields
tas_hist_mean = tas_hist.mean(dim='time')    # single historical mean
temperature map
tas_ssp_mean  = tas_ssp.mean(dim='time')     # single SSP245 mean
temperature map
pr_hist_mean  = pr_hist.mean(dim='time')     # single historical mean
precipitation map
pr_ssp_mean   = pr_ssp.mean(dim='time')      # single SSP245 mean
precipitation map

# Plot 7 - Historical Mean Temperature
spatial_plot(tas_hist_mean,
              'Historical Mean Temperature (1850-2014) - Canada',
              'RdYlBu_r', 'Temperature (°C)')
# RdYlBu_r: Red=warm, Yellow=moderate, Blue=cold (reversed so red=warm)

# Plot 8 - SSP245 Mean Temperature
spatial_plot(tas_ssp_mean,
              'SSP245 Mean Temperature (2015-2100) - Canada',
              'RdYlBu_r', 'Temperature (°C)')

# Plot 9 - Historical Mean Precipitation
spatial_plot(pr_hist_mean,
              'Historical Mean Precipitation (1850-2014) - Canada',
              'YlGnBu', 'Precipitation (mm/day)', vmin=0)
# YlGnBu: Yellow=dry, Green=moderate, Blue=wet; vmin=0 because rain ≥ 0

# Plot 10 - SSP245 Mean Precipitation
spatial_plot(pr_ssp_mean,
              'SSP245 Mean Precipitation (2015-2100) - Canada',
              'YlGnBu', 'Precipitation (mm/day)', vmin=0)

#
=====
====
# %% SECTION 13 - TREND COMPUTATION FUNCTION
#
=====
=====

```

```

#
# CONCEPT – LINEAR TREND AT EVERY GRID CELL
#
# A trend map answers: "At each location on the map, is the variable
# increasing or decreasing, and at what rate per decade?"
#
# Method for each individual grid cell (i, j):
# 1. Compute annual mean → one value per year (e.g. 165 values for
historical)
# 2. Fit a straight line through (year, annual_mean) using linear
regression
# 3. Slope of the line = rate of change per YEAR
# 4. Multiply slope × 10 → rate per DECADE (more intuitive to report)
# 5. p_value < 0.05 → the trend is "statistically significant"
# (less than 5% probability the observed change is due to random
noise)
#
# scipy.stats.linregress(x, y) returns:
# slope      = rate of change per unit x      (°C per year in our case)
# intercept  = y when x = 0                  (not used here)
# r_value    = correlation coefficient         (not used here)
# p_value    = statistical significance       (we use this for stippling
maps)
# std_err    = uncertainty of the slope       (not used here)

def compute_trend(da):
    """
    Compute the linear trend (per decade) at every grid cell
    independently.

    Parameters
    -----
    da : xarray DataArray with dimensions (time, lat, lon)
        Must have daily or sub-daily time steps – function first
        collapses to annual means before fitting.

    Returns
    -----
    trend_da : 2D DataArray – trend slope in [units per decade]
    pval_da  : 2D DataArray – p-value of the trend at each grid cell
    """
    # Step 1: get unique year values (the x-axis for the regression)
    years = da.time.dt.year.values # array of all years in the
time series
    unique_years = np.unique(years) # remove duplicates:
[1850, 1851, ...]

    # Step 2: collapse daily data to annual means (much faster than using
all days)
    annual = da.groupby('time.year').mean('time')
    # annual now has shape (n_years, n_lat, n_lon)

    # Step 3: prepare empty output arrays, filled with zeros initially

```

```

        trend = np.zeros((annual.shape[1], annual.shape[2]))    # shape:
(n_lat, n_lon)
        pval = np.zeros_like(trend)                                # same shape,
also zeros

        # Step 4: loop over every grid cell (this is the slow part)
        for i in range(annual.shape[1]):                            # loop over each latitude
row
            for j in range(annual.shape[2]):                        # loop over each longitude
column
                y = annual.values[:, i, j]                        # time series at this
single grid cell

                if np.all(np.isnan(y)):                            # skip cells that are all
NaN (e.g. ocean)
                    trend[i, j] = np.nan
                    pval[i, j] = np.nan
                else:
                    # scipy.stats.linregress fits  $y = \text{slope} \cdot x + \text{intercept}$ 
                    slope, _, _, p, _ = stats.linregress(unique_years, y)
                    trend[i, j] = slope * 10                        # convert °C/year →
°C/decade
                    pval[i, j] = p                                  # p-value for significance
test

        # Step 5: wrap results back into xarray DataArrays with proper
coordinates
        coords = {'lat': annual.lat, 'lon': annual.lon}
        trend_da = xr.DataArray(trend, dims=['lat', 'lon'], coords=coords)
        pval_da = xr.DataArray(pval, dims=['lat', 'lon'], coords=coords)

        return trend_da, pval_da

#
=====
====
# %% SECTION 14 – COMPUTE TRENDS (runs the function 4 times)
#
=====
====
# This section calls compute_trend() for all four combinations.
# ⚠ WARNING: This is the slowest part of the script.
# For 165 years × ~600 grid cells → expect 5-20 minutes total.
# Watch the progress messages in the Spyder console.

print("\nComputing spatial trends – please wait (5-20 minutes)...")

tas_hist_trend, tas_hist_pval = compute_trend(tas_hist)
print(" ✓ Temperature historical trend done")

tas_ssp_trend, tas_ssp_pval = compute_trend(tas_ssp)
print(" ✓ Temperature SSP245 trend done")

```

```

pr_hist_trend, pr_hist_pval = compute_trend(pr_hist)
print("  ✓ Precipitation historical trend done")

pr_ssp_trend, pr_ssp_pval = compute_trend(pr_ssp)
print("  ✓ Precipitation SSP245 trend done")
print("All trends computed.\n")

#
=====
====
# %% SECTION 15 – PLOTS 11-14: SPATIAL TREND MAPS
#
=====
====
#
# OUTPUT:
# Plot 11 – Historical temperature trend (°C/decade)
#           Red = warming, Blue = cooling.
#           Canada's Arctic regions often show the fastest warming.
# Plot 12 – SSP245 temperature trend (°C/decade)
#           Should show stronger/faster warming than historical.
# Plot 13 – Historical precipitation trend (mm/day per decade)
#           Brown = drying, Green = wetting.
# Plot 14 – SSP245 precipitation trend (mm/day per decade)
#
# vmin/vmax set symmetrically around 0 so that:
#   → zero change maps to white (centre of colourmap)
#   → positive change maps to one colour (e.g. red for warming)
#   → negative change maps to opposite colour (e.g. blue for cooling)

print("Generating Plots 11-14: Spatial trend maps")

spatial_plot(tas_hist_trend,
              'Historical Temperature Trend (°C/decade, 1850-2014) – Canada',
              'RdBu_r', '°C per decade',
              vmin=-0.5, vmax=0.5)
# RdBu_r: Red=positive(warming), White=zero, Blue=negative(cooling); _r = reversed

spatial_plot(tas_ssp_trend,
              'SSP245 Temperature Trend (°C/decade, 2015-2100) – Canada',
              'RdBu_r', '°C per decade',
              vmin=-0.5, vmax=0.5)

spatial_plot(pr_hist_trend,
              'Historical Precipitation Trend (mm/day per decade, 1850-2014) – Canada',
              'BrBG', 'mm/day per decade',
              vmin=-0.3, vmax=0.3)
# BrBG: Brown=drying, White=no change, Blue-Green=wetting

```

```

spatial_plot(pr_ssp_trend,
              'SSP245 Precipitation Trend (mm/day per decade, 2015-2100) -
Canada',
              'BrBG', 'mm/day per decade',
              vmin=-0.3, vmax=0.3)

```

```

#
=====
====
# %% SECTION 16 - PLOTS 15-16: TIME SERIES WITH TREND LINES
#
=====
====
#
# These plots combine the yearly time series (raw year-by-year values)
with
# a smoothed linear TREND LINE overlaid as a dashed line.
#
# np.polyfit(x, y, 1) → fits a degree-1 polynomial (straight line)
through
# the data using least-squares → returns [slope, intercept]
# np.polyval(coeffs, x) → evaluates the fitted line at every x point
# → gives the y-value on the trend line for each year
#
# The trend rate is shown in the legend (e.g. "+0.18 °C/decade").
# Positive = warming; negative = cooling.
#
# OUTPUT:
# Plot 15 - Temperature 1850-2100 with historical and SSP245 trend
lines.
#           You will see: historical warming is slower; SSP245 warming
faster.
# Plot 16 - Precipitation 1850-2100 with trend lines.

print("Generating Plots 15-16: Annual time series with trend lines")

# --- Fit linear trend lines for temperature ---
z_tas_hist = np.polyfit(tas_hist_yearly.year.values,
                        tas_hist_yearly.values, 1)
# z_tas_hist[0] = slope (°C/year), z_tas_hist[1] = intercept
z_tas_ssp = np.polyfit(tas_ssp_yearly.year.values,
                       tas_ssp_yearly.values, 1)

fig, ax = plt.subplots(figsize=(14, 5))
# Raw annual values (thin, semi-transparent)
ax.plot(tas_hist_yearly.year, tas_hist_yearly,
        label='Historical', color='steelblue', linewidth=0.8, alpha=0.7)
ax.plot(tas_ssp_yearly.year, tas_ssp_yearly,
        label='SSP245', color='tomato', linewidth=0.8, alpha=0.7)
# Trend lines (thick dashed)
ax.plot(tas_hist_yearly.year,
        np.polyval(z_tas_hist, tas_hist_yearly.year.values),
        '--', color='navy', linewidth=2.0,

```



```

        label=f"Hist trend: {z_tas_hist[0]*10:+.3f} °C/decade")
# {z_tas_hist[0]*10:+.3f} formats the slope×10 with sign (+/-) and 3
decimals
ax.plot(tas_ssp_yearly.year,
        np.polyval(z_tas_ssp, tas_ssp_yearly.year.values),
        '--', color='darkred', linewidth=2.0,
        label=f"SSP245 trend: {z_tas_ssp[0]*10:+.3f} °C/decade")
ax.set_xlabel('Year', fontsize=11)
ax.set_ylabel('Temperature (°C)', fontsize=11)
ax.set_title('Area-Averaged Annual Temperature – Canada with Trend
Lines', fontsize=13, fontweight='bold')
ax.legend(fontsize=9)
ax.grid(True, alpha=0.25)
plt.tight_layout()
# plt.savefig(r"E:\eco_data\plots\15_timeseries_trend_tas.png", dpi=150,
bbox_inches='tight')
plt.show()

```

```

# --- Fit linear trend lines for precipitation ---
z_pr_hist = np.polyfit(pr_hist_yearly.year.values,
                        pr_hist_yearly.values, 1)
z_pr_ssp   = np.polyfit(pr_ssp_yearly.year.values,
                        pr_ssp_yearly.values, 1)

```

```

fig, ax = plt.subplots(figsize=(14, 5))
ax.plot(pr_hist_yearly.year, pr_hist_yearly,
        label='Historical', color='steelblue', linewidth=0.8, alpha=0.7)
ax.plot(pr_ssp_yearly.year, pr_ssp_yearly,
        label='SSP245', color='tomato', linewidth=0.8, alpha=0.7)
ax.plot(pr_hist_yearly.year,
        np.polyval(z_pr_hist, pr_hist_yearly.year.values),
        '--', color='navy', linewidth=2.0,
        label=f"Hist trend: {z_pr_hist[0]*10:+.4f} mm/day per decade")
ax.plot(pr_ssp_yearly.year,
        np.polyval(z_pr_ssp, pr_ssp_yearly.year.values),
        '--', color='darkred', linewidth=2.0,
        label=f"SSP245 trend: {z_pr_ssp[0]*10:+.4f} mm/day per decade")
ax.set_xlabel('Year', fontsize=11)
ax.set_ylabel('Precipitation (mm/day)', fontsize=11)
ax.set_title('Area-Averaged Annual Precipitation – Canada with Trend
Lines', fontsize=13, fontweight='bold')
ax.legend(fontsize=9)
ax.grid(True, alpha=0.25)
plt.tight_layout()
# plt.savefig(r"E:\eco_data\plots\16_timeseries_trend_pr.png", dpi=150,
bbox_inches='tight')
plt.show()

```

```

#
=====
====
# %% SECTION 17 – PLOTS 17-18: SEASONAL CYCLE COMPARISON

```

```

#
=====
=====
#
# CONCEPT - SEASONS
# Canada's four meteorological seasons:
#   DJF = December, January, February      (winter)
#   MAM = March, April, May                 (spring)
#   JJA = June, July, August                (summer)
#   SON = September, October, November     (autumn)
#
# groupby('time.season') automatically groups data into these four
seasons
# and averages within each group across all years AND all grid cells.
#
# OUTPUT:
#   Plot 17 - Bar chart showing average temperature in each season.
#             DJF bars (winter) will be very negative for Canada.
#             JJA bars (summer) will be the warmest.
#             Are any seasons warming more than others in SSP245?
#   Plot 18 - Same for precipitation.
#             Which season sees the most precipitation change?

print("Generating Plots 17-18: Seasonal cycle comparison")

season_order = ['DJF', 'MAM', 'JJA', 'SON'] # fixed display order

# Average by season: result has dimension 'season' with 4 values
tas_hist_season = tas_hist.groupby('time.season').mean(dim=['time',
'lat', 'lon'])
tas_ssp_season = tas_ssp.groupby('time.season').mean(dim=['time', 'lat',
'lon'])

x = np.arange(4) # [0, 1, 2, 3] - positions for the 4 season groups on
x-axis

fig, ax = plt.subplots(figsize=(8, 5))
ax.bar(x - 0.15,
       [float(tas_hist_season.sel(season=s)) for s in season_order],
       # .sel(season=s) selects the value for season s; float() converts
to Python number
       0.3, label='Historical', color='steelblue', alpha=0.85,
edgecolor='white')
ax.bar(x + 0.15,
       [float(tas_ssp_season.sel(season=s)) for s in season_order],
       0.3, label='SSP245', color='tomato', alpha=0.85,
edgecolor='white')
ax.axhline(0, color='black', linewidth=0.5, alpha=0.4) # 0°C reference
line
ax.set_xticks(x)
ax.set_xticklabels(season_order, fontsize=11)
ax.set_ylabel('Temperature (°C)', fontsize=11)
ax.set_title('Seasonal Mean Temperature - Historical vs SSP245 (Canada)',
fontsize=13, fontweight='bold')

```

```

ax.legend(fontsize=10)
ax.grid(True, alpha=0.25, axis='y')
plt.tight_layout()
# plt.savefig(r"E:\eco_data\plots\17_seasonal_tas.png", dpi=150,
bbox_inches='tight')
plt.show()

# --- Seasonal Precipitation ---
pr_hist_season = pr_hist.groupby('time.season').mean(dim=['time', 'lat',
'lon'])
pr_ssp_season = pr_ssp.groupby('time.season').mean(dim=['time', 'lat',
'lon'])

fig, ax = plt.subplots(figsize=(8, 5))
ax.bar(x - 0.15,
      [float(pr_hist_season.sel(season=s)) for s in season_order],
      0.3, label='Historical', color='steelblue', alpha=0.85,
edgecolor='white')
ax.bar(x + 0.15,
      [float(pr_ssp_season.sel(season=s)) for s in season_order],
      0.3, label='SSP245', color='tomato', alpha=0.85,
edgecolor='white')
ax.set_xticks(x)
ax.set_xticklabels(season_order, fontsize=11)
ax.set_ylabel('Precipitation (mm/day)', fontsize=11)
ax.set_title('Seasonal Mean Precipitation - Historical vs SSP245
(Canada)', fontsize=13, fontweight='bold')
ax.legend(fontsize=10)
ax.grid(True, alpha=0.25, axis='y')
plt.tight_layout()
# plt.savefig(r"E:\eco_data\plots\18_seasonal_pr.png", dpi=150,
bbox_inches='tight')
plt.show()

```

```

#
=====
====
# %% SECTION 18 - PLOTS 19-20: ANOMALY DIFFERENCE MAPS
#
=====
====
#
# CONCEPT - ANOMALY (DIFFERENCE) MAP
# Simple subtraction: Future mean - Historical mean = Change
# A positive value at a grid cell means that location gets WARMER /
WETTER.
# A negative value means COOLER / DRIER.
# This is the most direct way to visualise WHERE and HOW MUCH Canada
changes.
#
# OUTPUT:
#   Plot 19 - Temperature change map (°C): SSP245 - Historical
#           Red regions = warming; Blue regions = cooling.

```

```

#           Canada's Arctic is expected to warm the most.
#   Plot 20 – Precipitation change map (mm/day): SSP245 – Historical
#           Brown = drying; Blue-Green = wetting.

print("Generating Plots 19-20: Anomaly/difference maps")

# Subtract: every grid cell (SSP245_mean) – (Historical_mean)
tas_diff = tas_ssp_mean – tas_hist_mean   # 2D array of temperature
change (°C)
pr_diff  = pr_ssp_mean  – pr_hist_mean    # 2D array of precipitation
change (mm/day)

spatial_plot(tas_diff,
             'Temperature Change: SSP245 – Historical (°C) – Canada',
             'RdBu_r', 'ΔT (°C)',
             vmin=-2, vmax=5, extend='both')
# vmin=-2, vmax=5: most of Canada warms 1-5°C; allows scale to show that

spatial_plot(pr_diff,
             'Precipitation Change: SSP245 – Historical (mm/day) –
Canada',
             'BrBG', 'Δpr (mm/day)',
             vmin=-2, vmax=2, extend='both')

#
=====
====
# %% SECTION 19 – PLOTS 21-22: BOX PLOTS OF DAILY DISTRIBUTIONS BY MONTH
#
=====
====
#
# CONCEPT – BOX PLOT
# A box plot shows the STATISTICAL DISTRIBUTION of daily values in each
month.
# For each month:
#   – The BOX spans the 25th to 75th percentile (middle 50% of all daily
values)
#   – The LINE inside the box = the median (50th percentile)
#   – The WHISKERS extend to the min/max within 1.5× the box height
#     (showfliers=False hides extreme outliers for cleaner comparison)
#
# We first convert to a pandas DataFrame so boxplot() can process it
easily.
# .to_dataframe() converts xarray → pandas table.
# .reset_index() moves the time coordinate from index to a regular
column.
#
# OUTPUT:
#   Plot 21 – Side-by-side: Historical (left) vs SSP245 (right)
temperature
#           distribution for each month.
#           January will show a wide spread (cold/warm days).

```

```

#           Has the distribution shifted upward in SSP245?
#   Plot 22 – Same for precipitation.

print("Generating Plots 21-22: Box plots of daily distributions")

# Collapse spatial dimensions to area-average first (one value per day)
# → then convert to DataFrame for easy monthly grouping
tas_hist_df = (tas_hist.mean(dim=['lat', 'lon'])    # average over all
Canada grid cells
                .to_dataframe()                    # convert xarray to
pandas table
                .reset_index())                    # move 'time' from
index to a column
tas_ssp_df = (tas_ssp.mean(dim=['lat', 'lon'])
               .to_dataframe()
               .reset_index())

# Add a 'month' column (integer 1-12) extracted from the 'time' column
tas_hist_df['month'] = tas_hist_df['time'].dt.month
tas_ssp_df['month'] = tas_ssp_df['time'].dt.month

# Build list of arrays: one array per month, containing all daily values
for that month
data_hist_tas = [tas_hist_df[tas_hist_df['month'] == m]['tas'].values for
m in range(1, 13)]
data_ssp_tas = [tas_ssp_df[tas_ssp_df['month'] == m]['tas'].values for
m in range(1, 13)]

fig, axes = plt.subplots(1, 2, figsize=(16, 6), sharey=True)
# sharey=True: both subplots share the same y-axis scale for direct
comparison

axes[0].boxplot(data_hist_tas, labels=months, showfliers=False)
# showfliers=False: hides individual outlier dots for a cleaner plot
axes[0].set_title('Historical – Daily Temperature Distribution',
fontsize=12, fontweight='bold')
axes[0].set_ylabel('Temperature (°C)', fontsize=11)
axes[0].grid(True, alpha=0.25, axis='y')

axes[1].boxplot(data_ssp_tas, labels=months, showfliers=False)
axes[1].set_title('SSP245 – Daily Temperature Distribution', fontsize=12,
fontweight='bold')
axes[1].grid(True, alpha=0.25, axis='y')

plt.suptitle('Box Plots of Daily Temperature by Month (Area-Averaged,
Canada)',
             fontsize=14, fontweight='bold')
plt.tight_layout()
# plt.savefig(r"E:\eco_data\plots\21_boxplot_tas.png", dpi=150,
bbox_inches='tight')
plt.show()

# --- Box Plots – Precipitation ---

```

```

pr_hist_df = (pr_hist.mean(dim=['lat',
'lon']).to_dataframe().reset_index())
pr_ssp_df = (pr_ssp.mean(dim=['lat',
'lon']).to_dataframe().reset_index())
pr_hist_df['month'] = pr_hist_df['time'].dt.month
pr_ssp_df['month'] = pr_ssp_df['time'].dt.month

data_hist_pr = [pr_hist_df[pr_hist_df['month'] == m]['pr'].values for m
in range(1, 13)]
data_ssp_pr = [pr_ssp_df[pr_ssp_df['month'] == m]['pr'].values for m
in range(1, 13)]

fig, axes = plt.subplots(1, 2, figsize=(16, 6), sharey=True)
axes[0].boxplot(data_hist_pr, labels=months, showfliers=False)
axes[0].set_title('Historical - Daily Precipitation Distribution',
fontsize=12, fontweight='bold')
axes[0].set_ylabel('Precipitation (mm/day)', fontsize=11)
axes[0].grid(True, alpha=0.25, axis='y')
axes[1].boxplot(data_ssp_pr, labels=months, showfliers=False)
axes[1].set_title('SSP245 - Daily Precipitation Distribution',
fontsize=12, fontweight='bold')
axes[1].grid(True, alpha=0.25, axis='y')
plt.suptitle('Box Plots of Daily Precipitation by Month (Area-Averaged,
Canada)',
            fontsize=14, fontweight='bold')
plt.tight_layout()
# plt.savefig(r"E:\eco_data\plots\22_boxplot_pr.png", dpi=150,
bbox_inches='tight')
plt.show()

#
=====
====
# %% SECTION 20 - PLOTS 23-24: PROBABILITY DENSITY FUNCTIONS (PDF)
#
=====
====
#
# CONCEPT - PDF / HISTOGRAM
# A PDF shows the FULL DISTRIBUTION of all daily values across all years
and
# all grid cells. It answers: "How often does Canada experience each
temperature?"
#
# .flatten() converts the multi-dimensional array into a 1-D list of all
values.
# ~np.isnan() removes NaN values before plotting.
# ax.hist(..., density=True) draws a histogram normalised so the total
area = 1
# → this makes it a "probability density" curve (shape is what
matters).
# alpha=0.5 → 50% transparent so the two overlapping histograms are both
visible.

```

```

#
# For precipitation: we clip to the 99th percentile to avoid extreme
outliers
#   from stretching the x-axis (rare very high rainfall events).
#
# OUTPUT:
#   Plot 23 – Two overlapping temperature distributions.
#               If SSP245 curve shifts RIGHT, the whole distribution has
warmed.
#               If it also gets WIDER, there is more variability.
#   Plot 24 – Two overlapping precipitation distributions.
#               Changes in the tail (right side) indicate changing extreme
events.

```

```

print("Generating Plots 23-24: Probability density functions")

```

```

# Temperature PDF
tas_hist_flat = tas_hist.mean(dim=['lat', 'lon']).values.flatten() #
all daily values → 1D
tas_ssp_flat = tas_ssp.mean(dim=['lat', 'lon']).values.flatten()
tas_hist_flat = tas_hist_flat[~np.isnan(tas_hist_flat)] # remove NaN
values
tas_ssp_flat = tas_ssp_flat[~np.isnan(tas_ssp_flat)]

```

```

fig, ax = plt.subplots(figsize=(10, 5))
ax.hist(tas_hist_flat, bins=80, density=True, alpha=0.5,
        label='Historical (1850-2014)', color='steelblue')
ax.hist(tas_ssp_flat, bins=80, density=True, alpha=0.5,
        label='SSP245 (2015-2100)', color='tomato')
ax.set_xlabel('Temperature (°C)', fontsize=11)
ax.set_ylabel('Probability Density', fontsize=11)
ax.set_title('PDF – Daily Temperature Distribution (Area-Averaged,
Canada)', fontsize=13, fontweight='bold')
ax.legend(fontsize=10)
ax.grid(True, alpha=0.25)
plt.tight_layout()
# plt.savefig(r"E:\eco_data\plots\23_pdf_tas.png", dpi=150,
bbox_inches='tight')
plt.show()

```

```

# Precipitation PDF
pr_hist_flat = pr_hist.mean(dim=['lat', 'lon']).values.flatten()
pr_ssp_flat = pr_ssp.mean(dim=['lat', 'lon']).values.flatten()
pr_hist_flat = pr_hist_flat[~np.isnan(pr_hist_flat)]
pr_ssp_flat = pr_ssp_flat[~np.isnan(pr_ssp_flat)]

fig, ax = plt.subplots(figsize=(10, 5))
ax.hist(pr_hist_flat, bins=80, density=True, alpha=0.5,
        label='Historical (1850-2014)', color='steelblue',
        range=(0, np.percentile(pr_hist_flat, 99)))
# range=(0, 99th percentile): excludes the top 1% extreme values for
cleaner axis
ax.hist(pr_ssp_flat, bins=80, density=True, alpha=0.5,
        label='SSP245 (2015-2100)', color='tomato',

```


Draw a spatial trend map with black dot stippling for significant cells.

```
Parameters
-----
trend_da    : 2D DataArray - trend values (output of compute_trend)
pval_da     : 2D DataArray - p-values (output of compute_trend)
title       : String - figure title
cmap        : Colormap name
cbar_label  : String - colour bar label
vmin, vmax  : Colour scale limits
"""
fig, ax = plt.subplots(
    figsize=(11, 7),
    subplot_kw={'projection': ccrs.PlateCarree()}
)

# Draw the trend colour map
trend_da.plot(
    ax=ax,
    transform=ccrs.PlateCarree(),
    cmap=cmap,
    vmin=vmin, vmax=vmax,
    add_colorbar=True,
    cbar_kwargs={'label': cbar_label, 'shrink': 0.65, 'extend':
'both', 'pad': 0.02}
)

# --- STIPPLING: mark significant cells with black dots ---
# Keep only cells where p < 0.05; all others become NaN
sig = pval_da.where(pval_da < 0.05)

# Create 2D meshgrids of all longitude and latitude combinations
lons, lats = np.meshgrid(sig.lon.values, sig.lat.values)
# lons, lats now both have shape (n_lat, n_lon)

# Boolean mask: True wherever the p-value was significant (not NaN)
mask = ~np.isnan(sig.values)

# Draw tiny black dots at significant grid cells
ax.scatter(
    lons[mask], lats[mask],          # x and y positions of significant
cells
    marker='.', s=3,                # dot shape and size (3 pt2)
    color='black',
    transform=ccrs.PlateCarree(),  # data coordinates are lat/lon
degrees
    label='p < 0.05 (significant)'
)

# Geographic features
ax.add_feature(cfeature.COASTLINE, linewidth=0.8)
ax.add_feature(cfeature.BORDERS,   linewidth=0.7, linestyle='--')
ax.add_feature(cfeature.STATES,    linewidth=0.3, linestyle=':')
```

```

ax.add_feature(cfeature.LAKES, linewidth=0.4, alpha=0.5)
ax.set_extent(extent_cartopy, crs=ccrs.PlateCarree())

gl = ax.gridlines(draw_labels=True, linewidth=0.3, alpha=0.5,
                  linestyle='--', color='gray')
gl.top_labels = False
gl.right_labels = False
gl.xlocator = mticker.FixedLocator([-140, -120, -100, -80, -60])
gl.ylocator = mticker.FixedLocator([45, 55, 65, 75])

ax.set_title(title, fontsize=12, fontweight='bold', pad=12)
ax.legend(loc='lower left', fontsize=9, markerscale=4)
plt.tight_layout()
plt.show()

# Plot 25 – Historical Temperature Trend + Significance
plot_trend_significance(
    tas_hist_trend, tas_hist_pval,
    'Historical Temperature Trend (°C/decade) with Significance –
Canada\n'
    '(Black dots = statistically significant at  $p < 0.05$ )',
    'RdBu_r', '°C per decade', vmin=-0.5, vmax=0.5
)

# Plot 26 – SSP245 Temperature Trend + Significance
plot_trend_significance(
    tas_ssp_trend, tas_ssp_pval,
    'SSP245 Temperature Trend (°C/decade) with Significance – Canada\n'
    '(Black dots = statistically significant at  $p < 0.05$ )',
    'RdBu_r', '°C per decade', vmin=-0.5, vmax=0.5
)

# Plot 27 – Historical Precipitation Trend + Significance
plot_trend_significance(
    pr_hist_trend, pr_hist_pval,
    'Historical Precipitation Trend (mm/day per decade) with Significance
– Canada\n'
    '(Black dots = statistically significant at  $p < 0.05$ )',
    'BrBG', 'mm/day per decade', vmin=-0.3, vmax=0.3
)

# Plot 28 – SSP245 Precipitation Trend + Significance
plot_trend_significance(
    pr_ssp_trend, pr_ssp_pval,
    'SSP245 Precipitation Trend (mm/day per decade) with Significance –
Canada\n'
    '(Black dots = statistically significant at  $p < 0.05$ )',
    'BrBG', 'mm/day per decade', vmin=-0.3, vmax=0.3
)

#
=====
=====

```

```

# %% SECTION 22 - PLOTS 29-30: BASELINE vs END-CENTURY SIDE-BY-SIDE MAPS
#
=====
====
#
# CONCEPT - PERIOD COMPARISON
# Instead of comparing the ENTIRE historical period vs ENTIRE future
period
# (as in Plots 7-10), here we compare two specific 30-year windows:
#
#   Baseline      = 1985-2014   (last 30 years of historical data)
#                   This is the "recent past" reference period.
#   End-century   = 2071-2100   (last 30 years of SSP245 data)
#                   This is the "far future" - what Canada might look like
by 2100.
#
# 30-year periods are the WMO (World Meteorological Organisation)
standard
# for computing climate normals - long enough to average out year-to-year
# variability but short enough to represent a specific time period.
#
# .sel(time=slice('1985','2014')) selects only the years 1985 to 2014.
# .mean(dim='time') then averages over those 30 years.
#
# OUTPUT:
#   Plot 29 - Side-by-side temperature: 1985-2014 (left) vs 2071-2100
(right)
#           Direct visual comparison of how warm Canada was vs will be.
#   Plot 30 - Same for precipitation.

print("Generating Plots 29-30: Baseline vs end-century comparisons")

# Compute 30-year period means
tas_hist_last30 = tas_hist.sel(time=slice('1985',
'2014')).mean(dim='time')
# .sel(time=slice('1985','2014')) → keeps only data from 1985-01-01 to
2014-12-31
# .mean(dim='time') → average over those 30 years → 2D map
tas_ssp_last30 = tas_ssp.sel(time=slice('2071',
'2100')).mean(dim='time')
pr_hist_last30 = pr_hist.sel(time=slice('1985',
'2014')).mean(dim='time')
pr_ssp_last30 = pr_ssp.sel(time=slice('2071', '2100')).mean(dim='time')

# --- Side-by-side Temperature ---
fig, axes = plt.subplots(
    1, 2,                                # 1 row, 2 columns (two maps next
to each other)
    figsize=(18, 7),
    subplot_kw={'projection': ccrs.PlateCarree()})

for ax, data, title in zip(
    axes,

```

```

[ tas_hist_last30, tas_ssp_last30 ],
[ 'Baseline Temperature 1985-2014', 'End-Century Temperature 2071-2100
(SSP245)' ]
):
    data.plot(ax=ax, transform=ccrs.PlateCarree(),
              cmap='RdYlBu_r', add_colorbar=True,
              cbar_kwargs={'label': '°C', 'shrink': 0.7, 'pad': 0.02})
    ax.add_feature(cfeature.COASTLINE, linewidth=0.8)
    ax.add_feature(cfeature.BORDERS,   linewidth=0.7, linestyle='--')
    ax.add_feature(cfeature.STATES,    linewidth=0.3, linestyle=':')
    ax.add_feature(cfeature.LAKES,     linewidth=0.4, alpha=0.4)
    ax.set_extent(extent_cartopy, crs=ccrs.PlateCarree())
    gl = ax.gridlines(draw_labels=True, linewidth=0.3, alpha=0.5)
    gl.top_labels = False
    gl.right_labels = False
    ax.set_title(title, fontsize=12, fontweight='bold')

plt.suptitle('Mean Temperature: Recent Baseline vs End-Century - Canada',
             fontsize=14, fontweight='bold')
plt.tight_layout()
# plt.savefig(r"E:\eco_data\plots\29_sidebyside_tas.png", dpi=150,
#             bbox_inches='tight')
plt.show()

# --- Side-by-side Precipitation ---
fig, axes = plt.subplots(1, 2, figsize=(18, 7),
                        subplot_kw={'projection': ccrs.PlateCarree()})

for ax, data, title in zip(
    axes,
    [pr_hist_last30, pr_ssp_last30],
    [ 'Baseline Precipitation 1985-2014', 'End-Century Precipitation 2071-
2100 (SSP245)' ]
):
    data.plot(ax=ax, transform=ccrs.PlateCarree(),
              cmap='YlGnBu', add_colorbar=True,
              cbar_kwargs={'label': 'mm/day', 'shrink': 0.7, 'pad':
0.02})
    ax.add_feature(cfeature.COASTLINE, linewidth=0.8)
    ax.add_feature(cfeature.BORDERS,   linewidth=0.7, linestyle='--')
    ax.add_feature(cfeature.STATES,    linewidth=0.3, linestyle=':')
    ax.add_feature(cfeature.LAKES,     linewidth=0.4, alpha=0.4)
    ax.set_extent(extent_cartopy, crs=ccrs.PlateCarree())
    gl = ax.gridlines(draw_labels=True, linewidth=0.3, alpha=0.5)
    gl.top_labels = False
    gl.right_labels = False
    ax.set_title(title, fontsize=12, fontweight='bold')

plt.suptitle('Mean Precipitation: Recent Baseline vs End-Century -
Canada',
             fontsize=14, fontweight='bold')
plt.tight_layout()
# plt.savefig(r"E:\eco_data\plots\30_sidebyside_pr.png", dpi=150,
#             bbox_inches='tight')

```

```
plt.show()
```

```
#
=====
====
# %% SECTION 23 – PLOTS 31-32: MOVING AVERAGE TIME SERIES
#
=====
====
#
# CONCEPT – MOVING AVERAGE (ROLLING MEAN)
# Year-by-year values are noisy – a single volcanic eruption, El Niño
event,
# or unusual winter can make one year much colder than its neighbours.
# A 10-year MOVING AVERAGE smooths out this year-to-year noise to reveal
# the underlying MULTI-DECADAL TREND more clearly.
#
# How it works:
#   For a 10-year window centred on year Y:
#     Average value of Y-4, Y-3, ..., Y, ..., Y+4, Y+5 (10 years)
#   → As Y advances, the window "rolls" forward → "rolling" or "moving"
average
#
# .rolling(year=10, center=True) → creates 10-year rolling windows,
centred
# .mean() → computes the mean within each window
# The first and last 5 years will be NaN (window goes outside the data
range)
#
# OUTPUT:
#   Plot 31 – Thin lines = raw annual temperature; thick lines = 10-year
MA.
#           The thick lines show the real long-term warming signal.
#   Plot 32 – Same for precipitation.
#           Any long-term drying or wetting trend becomes clearly
visible.

print("Generating Plots 31-32: Moving average time series")

window = 10    # number of years in the rolling window

# Compute 10-year rolling mean on the already-computed annual series
tas_hist_ma = tas_hist_yearly.rolling(year=window, center=True).mean()
tas_ssp_ma  = tas_ssp_yearly.rolling(year=window, center=True).mean()
# .rolling(year=window, center=True): create rolling windows of 'window'
years
# .mean(): compute the mean of each window

fig, ax = plt.subplots(figsize=(14, 5))
# Raw annual values (thin, 30% opaque so they don't dominate the plot)
ax.plot(tas_hist_yearly.year, tas_hist_yearly,
        alpha=0.3, color='steelblue', linewidth=0.8)
ax.plot(tas_ssp_yearly.year, tas_ssp_yearly,
```

```

        alpha=0.3, color='tomato',    linewidth=0.8)
# 10-year moving average (thick, fully opaque – this is the key line)
ax.plot(tas_hist_ma.year, tas_hist_ma,
        color='steelblue', linewidth=2.5,
        label=f'Historical ({window}-yr moving avg)')
ax.plot(tas_ssp_ma.year, tas_ssp_ma,
        color='tomato',    linewidth=2.5,
        label=f'SSP245 ({window}-yr moving avg)')
ax.set_xlabel('Year', fontsize=11)
ax.set_ylabel('Temperature (°C)', fontsize=11)
ax.set_title(f'Annual Temperature with {window}-Year Moving Average –
Canada',
            fontsize=13, fontweight='bold')
ax.legend(fontsize=10)
ax.grid(True, alpha=0.25)
plt.tight_layout()
# plt.savefig(r"E:\eco_data\plots\31_movingavg_tas.png", dpi=150,
bbox_inches='tight')
plt.show()

# --- Moving Average Precipitation ---
pr_hist_ma = pr_hist_yearly.rolling(year=window, center=True).mean()
pr_ssp_ma  = pr_ssp_yearly.rolling(year=window, center=True).mean()

fig, ax = plt.subplots(figsize=(14, 5))
ax.plot(pr_hist_yearly.year, pr_hist_yearly,
        alpha=0.3, color='steelblue', linewidth=0.8)
ax.plot(pr_ssp_yearly.year, pr_ssp_yearly,
        alpha=0.3, color='tomato',    linewidth=0.8)
ax.plot(pr_hist_ma.year, pr_hist_ma,
        color='steelblue', linewidth=2.5,
        label=f'Historical ({window}-yr moving avg)')
ax.plot(pr_ssp_ma.year, pr_ssp_ma,
        color='tomato',    linewidth=2.5,
        label=f'SSP245 ({window}-yr moving avg)')
ax.set_xlabel('Year', fontsize=11)
ax.set_ylabel('Precipitation (mm/day)', fontsize=11)
ax.set_title(f'Annual Precipitation with {window}-Year Moving Average –
Canada',
            fontsize=13, fontweight='bold')
ax.legend(fontsize=10)
ax.grid(True, alpha=0.25)
plt.tight_layout()
# plt.savefig(r"E:\eco_data\plots\32_movingavg_pr.png", dpi=150,
bbox_inches='tight')
plt.show()

```

```
#
```

```
=====
=====
```

```
# %% SECTION 24 – COMPLETION SUMMARY
```

```

#
=====

print("\n" + "=" * 65)
print("  ✓  ALL 32 PLOTS GENERATED SUCCESSFULLY")
print("=" * 65)
print("""
PLOTS PRODUCED (in order):


---


Mandatory – Climatology:
01  Daily climatology          – Temperature
02  Daily climatology          – Precipitation
03  Monthly climatology        – Temperature
04  Monthly climatology        – Precipitation
05  Yearly time series         – Temperature
06  Yearly time series         – Precipitation

Mandatory – Spatial trends:
07  Spatial mean map           – Historical Temperature
08  Spatial mean map           – SSP245 Temperature
09  Spatial mean map           – Historical Precipitation
10  Spatial mean map           – SSP245 Precipitation
11  Spatial trend map          – Historical Temperature (°C/decade)
12  Spatial trend map          – SSP245 Temperature (°C/decade)
13  Spatial trend map          – Historical Precipitation
(mm/day/decade)
14  Spatial trend map          – SSP245 Precipitation (mm/day/decade)

Additional – Supporting analysis:
15  Annual time series + trend lines – Temperature
16  Annual time series + trend lines – Precipitation
17  Seasonal comparison          – Temperature
18  Seasonal comparison          – Precipitation
19  Anomaly/difference map       – Temperature change
20  Anomaly/difference map       – Precipitation change
21  Box plots by month           – Temperature
22  Box plots by month           – Precipitation
23  PDF histogram                – Temperature
24  PDF histogram                – Precipitation
25  Trend + significance stippling – Historical Temperature
26  Trend + significance stippling – SSP245 Temperature
27  Trend + significance stippling – Historical Precipitation
28  Trend + significance stippling – SSP245 Precipitation
29  Baseline vs end-century map   – Temperature
30  Baseline vs end-century map   – Precipitation
31  Moving average time series    – Temperature
32  Moving average time series    – Precipitation


---



TO SAVE PLOTS: uncomment the plt.savefig(...) line just above
each plt.show() and change the path to your desired folder.
""")

```

```

#
=====
=====
#
=====
=====
#
# EXPLANATION OF WHAT THIS CODE DOES AND WHAT YOU WILL GET AS OUTPUT
# =====
#
# PURPOSE
# =====
# This script analyses 250+ years of climate model output for Canada
# (1850-2100) to answer two core questions:
#   1. How has Canada's climate varied historically?
#   2. How is it projected to change under the SSP245 (moderate
emissions)
#       scenario?
#
# DATA USED
# =====
# Model      : IPSL-CM6A-LR (French climate model, CMIP6)
# Variables  : tas (near-surface air temperature, 2 m)
#              pr  (daily precipitation)
# Periods    : Historical 1850-2014 | SSP245 2015-2100
# Region     : Canada (42°N-84°N, 52°W-141°W)
#

```